

Q.NO :8

Question description

Selvon is very interested in surfing the contents from google. He searches for various coding test on Google.

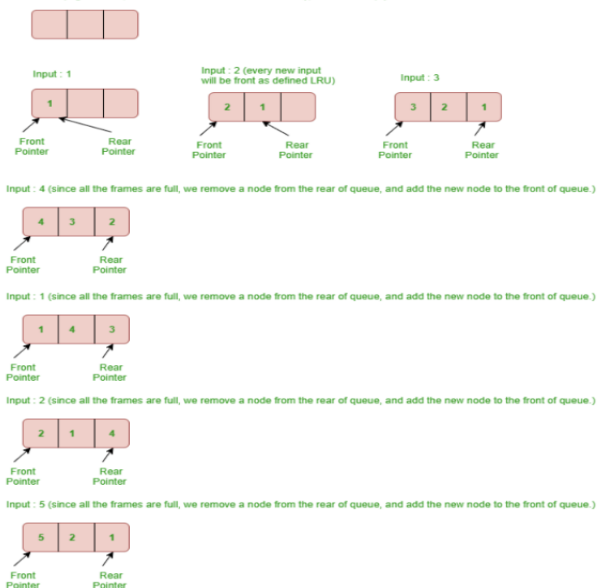
One day he searched about online coding competitions, in the retrieval links, he received many links for coding competition. he chooses first link from the google suggestion list.

first question for the coding competition is LRU cache implementation using queue concepts.

Function Description

1. **Queue** which is implemented using a doubly linked list. The maximum size of the queue will be equal to the total number of frames available (cache size). The most recently used pages will be near front end and least recently pages will be near the rear end.
2. **A Hash** with page number as key and address of the corresponding queue node as value.

Given 3 page frames, so we take size of Queue is 3. Initially, Queue is empty.



Constraints

For this experiment, cache can hold 4 pages.

Let 10 different pages can be requested (pages to be referenced are numbered from 0 to 9).

Input Format:

First line represents n and m, where n is the page number with in the range [0-9] & m is cache size (must be 4 for this problem).

Next line represents the reference pages.

Output Format:

Single line represents the cache frames after the above referenced pages.

Logical Test Cases

Test Case 1

INPUT (STDIN)

6 4
1 2 3 1 4 5

EXPECTED OUTPUT

5 4 1 3

Test Case 2

INPUT (STDIN)

10 4
3 1 4 5 9 8 7 6 1 2

EXPECTED OUTPUT

2 1 6 7

Mandatory Test Cases

Test Case 1

KEYWORD

QNode* newNode(unsigned pageNumber)

Test Case 4

KEYWORD

QNode* newNode(unsigned pageNumber)

Test Case 2

KEYWORD

Queue* createQueue(int numberOfFrames)

Test Case 3

KEYWORD

typedef struct Queue

Code:

```
#include <stdio.h>
```

```
#include <stdlib.h>

typedef struct QNode
{
    unsigned pageNumber;
    struct QNode *prev;
    struct QNode *next;
} QNode;

typedef struct Queue
{
    int numberOfFrames;
    int count;
    QNode *front;
    QNode *rear;
} Queue;

QNode* newQNode(unsigned pageNumber)
{
    QNode *temp;

    temp = (QNode*)malloc(sizeof(QNode));

    temp->pageNumber = pageNumber;
    temp->prev = NULL;
    temp->next = NULL;

    return temp;
}

Queue* createQueue(int numberOfFrames)
{
    Queue *queue;

    queue = (Queue*)malloc(sizeof(Queue));

    queue->numberOfFrames = numberOfFrames;
    queue->count = 0;
    queue->front = NULL;
    queue->rear = NULL;

    return queue;
}

void moveToFront(Queue *queue, QNode *node)
{
    if(queue->front == node)
        return;

    if(node->prev != NULL)
        node->prev->next = node->next;
```

```
if(node->next != NULL)
node->next->prev = node->prev;

if(queue->rear == node)
queue->rear = node->prev;

node->next = queue->front;
node->prev = NULL;

if(queue->front != NULL)
queue->front->prev = node;

queue->front = node;

if(queue->rear == NULL)
queue->rear = node;
}

void referPage(Queue *queue, unsigned pageNumber)
{
QNode *temp;

temp = queue->front;

while(temp != NULL)
{
if(temp->pageNumber == pageNumber)
{
moveToFront(queue, temp);
return;
}

temp = temp->next;
}

if(queue->count == queue->numberOfFrames)
{
temp = queue->rear;

if(temp->prev != NULL)
temp->prev->next = NULL;

queue->rear = temp->prev;

free(temp);

queue->count--;
}
```

```
temp = newQNode(pageNumber);

temp->next = queue->front;

if(queue->front != NULL)
queue->front->prev = temp;

queue->front = temp;

if(queue->rear == NULL)
queue->rear = temp;

queue->count++;
}

void printQueue(Queue *queue)
{
QNode *temp;

temp = queue->front;

while(temp != NULL)
{
printf("%u ", temp->pageNumber);
temp = temp->next;
}

printf("\n");
}

int main()
{
int n, size;
unsigned x;

printf("Input value:\n");

scanf("%d %d", &n, &size);

Queue *queue = createQueue(size);

for(int i = 0; i < n; i++)
{
scanf("%u", &x);
referPage(queue, x);
}

printf("\nOutput:\n");

printQueue(queue);
```

Aaditya Jaiswal

CH.SC.U4CSE25276

```
return 0;  
}
```

Output:

Test_case 1:

```
6 4  
1 2 3 1 4 5  
5 4 1 3 %c
```

Test_case 2:

```
10 4  
3 1 4 5 9 8 7 6 1 2  
2 1 6 7 %c
```